

Trust the Model Where It Trusts Itself

MACURA: the paper, and my project on a racing drone

Silver Gjeka

Department of Computer Science
Master's Degree in Artificial Intelligence
University of Verona

September 30, 2026

Paper: B. Frauenknecht, A. Eisele, D. Subhasish, F. Solowjow, S. Trimpe (RWTH Aachen), ICML 2024,
arXiv:2405.19014

Project code: github.com/silvergjeka22/macura-drone

Outline

Part I – The paper

1. The problem: when and where to trust a model
2. Background: SAC, ensembles, MBPO and M2AC
3. MACURA: uncertainty-aware rollouts
4. The four algorithms side by side
5. Noise: exploration and the environment
6. Results on MuJoCo

Part II – My project

7. The task: a racing drone with wind and a chute
8. Implementation and a fair protocol
9. Results: learning, testing, trust
10. The experiments I ran
11. Limitations, lessons and conclusion

Part I

The Paper

How MACURA decides *where* to trust its learned model

The Problem: When and Where to Trust a Model

- Deep RL needs **millions of interactions**; on real systems (robots, drones) each one is slow, expensive or risky.
- **Model-based RL (MBRL)** learns a model of the environment and **imagines** extra experience with it.
- But the model is never perfect: training on its **wrong predictions** misleads the agent.

MBPO: *when* to trust the model

Imagined rollouts get longer during training, following a **hand-designed schedule**, and the model is trusted **equally everywhere**.

Contributions: a monotonic-improvement guarantee on a set $\mathcal{E}$ of accurate states; a cheap uncertainty measure (GJS) to build $\mathcal{E}$; adaptive rollouts with essentially **one hyperparameter** ξ ; better results than MBPO and M2AC on MuJoCo.

MACURA: *where* to trust the model

Accuracy is **local**: every imagined rollout **stops where the model itself becomes uncertain**.

Background: SAC and Probabilistic Ensembles

Soft Actor-Critic (SAC): the learner inside every method

- Off-policy actor-critic: learns from a **replay buffer**.
- **Maximum entropy:** reward plus randomness, so it keeps exploring:

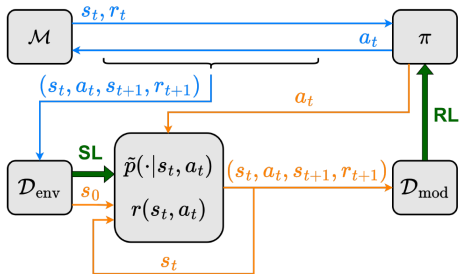
$$J(\pi) = \sum_t \mathbb{E}[r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))]$$

- Twin critics $Q_{\theta_1}, Q_{\theta_2}$ (take the smaller), a Gaussian actor π_ϕ , automatic temperature α .
- One real step $\Rightarrow$ G gradient updates (the **update-to-data ratio**).

Probabilistic ensemble (PE): the learned model

- $E = 7$ neural networks, each trained on its own bootstrap sample of the real data.
- Each predicts a **Gaussian**:
 $\tilde{p}_{\theta_e}(s' | s, a) = \mathcal{N}(\mu_{\theta_e}, \Sigma_{\theta_e})$.
- **Aleatoric** uncertainty (noise of the world) $\Rightarrow$ wide Σ_{θ_e} .
- **Epistemic** uncertainty (too little data) $\Rightarrow$ the members **disagree**.
- In a rollout, each step uses a **random member**.

Dyna-Style MBRL: Imagined Rollouts



Dyna-style MBRL (paper, Fig. 1).

- The agent acts in the real world $\Rightarrow$ real buffer $\mathcal{D}_{\text{env}}$; the model $\tilde{p}$ learns from $\mathcal{D}_{\text{env}}$ (supervised learning).
- **Branched rollouts:** start from real states $s_0 \in \mathcal{D}_{\text{env}}$ and imagine a few steps with π and $\tilde{p} \Rightarrow$ model buffer $\mathcal{D}_{\text{mod}}$.
- SAC trains mostly on $\mathcal{D}_{\text{mod}}$ (95% imagined + 5% real), with many updates per real step.

Why it helps: much more training data per real step, and it follows the **current** policy.

The risk: longer rollouts drift away from the real data and errors add up; the agent learns to exploit the model's mistakes (**model exploitation**).

$\Rightarrow$ The methods differ in **how many imagined steps they keep, and which**.

MBPO and M2AC: Two Earlier Answers

MBPO (Janner et al., 2019)

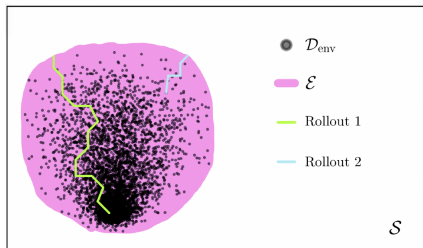
- Every rollout has the **same length** k , which grows during training by a **schedule** chosen by hand per task (Humanoid 1 $\rightarrow$ 25, Walker 1).
- **Every** imagined step is used: no check of the model.
- A **fixed** number G of SAC updates per real step.

$\Rightarrow$ answers *when*, not *where*.

M2AC (Pan et al., 2020): masking by uncertainty

- **One-vs-rest (OvR)** uncertainty: KL divergence between the member that made the prediction and a Gaussian fitted to the other members.
- All rollouts run the full horizon H ; at step h only the share $\frac{H-h}{2(H+1)}$ with the **lowest** uncertainty is kept ($\approx 25\%$ overall for $H = 10$).
- **Rank-based**: a fixed share is kept even when all data are bad, or thrown away when all are good.
- **Reward penalty**: stored reward $r - \alpha u$ pushes the agent away from uncertain regions, which can also slow exploration.

Where to Trust Your Model?



The real data define a set $\mathcal{E}$ where the model is accurate (paper, Fig. 2).

- Inside $\mathcal{E}$ longer rollouts are useful; once a rollout **leaves** $\mathcal{E}$ its data are harmful.
- Some rollouts stay in $\mathcal{E}$ for long, others leave it at once: **a fixed length cannot fit both.**

Theorem 4.1 (simplified): rollouts cut when they leave $\mathcal{E}$ give

$$\eta[\pi] \geq \tilde{\eta}[\pi] - C, \quad C = 2r_{\max} \sum_t \gamma^t \sum_{\tau \leq t} \Delta p_{\mathcal{E}}[\pi],$$

$\Delta p_{\mathcal{E}}$ = the largest model error **inside** $\mathcal{E}$. If the policy improves in the model by more than C , it improves in the real world.

In practice: the true error is unknown, so $\mathcal{E}$ is built from the model's own uncertainty:

$$\mathcal{E} = \{s \mid u(s, a) < \kappa, a \sim \pi(\cdot \mid s)\}.$$

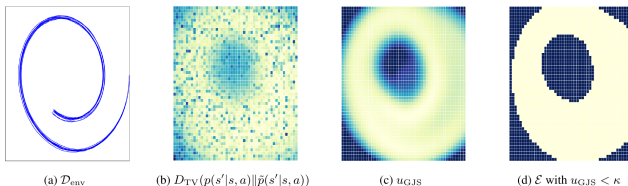
GJS Uncertainty: Do the Members Agree?

Average disagreement over all **pairs** of members ($\mathcal{N}_e$ = member e 's Gaussian):

$$u_{\text{GJS}}(s, a) = \frac{2}{E(E-1)} \sum_{e < f} D_{\text{GJS}}(\mathcal{N}_e \| \mathcal{N}_f), \quad D_{\text{GJS}} = \frac{1}{2} D_{\text{KL}}(\mathcal{N}_e \| \mathcal{N}_{ef}) + \frac{1}{2} D_{\text{KL}}(\mathcal{N}_f \| \mathcal{N}_{ef}),$$

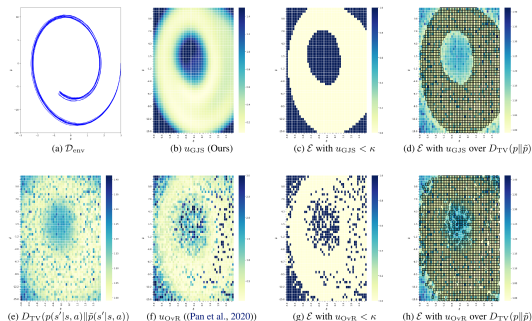
with $\mathcal{N}_{ef}$ the **geometric mean** of the two Gaussians (again a Gaussian).

- **Closed form** and symmetric: cheap enough for **every imagined step**.
- Large when two members are **confident but predict different things**.



Pendulum (paper, Fig. 3): (a) data, (b) true model error, (c) u_{GJS} , (d) the set $\mathcal{E}$. u_{GJS} is low exactly where the real error is low.

Why a New Measure? GJS (MACURA) vs OvR (M2AC)



Top: GJS. Bottom: OvR. On the pendulum data (paper, Fig. 10).

- **GJS** (b) is smooth and follows the data; its set $\mathcal{E}$ (c) has a clear border and matches the regions of **low real error** (d).
- **OvR** (f) is **noisy** and **over-confident**: low uncertainty in the centre, where the real error is high (e). Its set (g) does not match the accurate region (h).
- OvR looks at **one** member against the rest; GJS averages over **all pairs**, so one lucky member cannot hide the disagreement.

⇒ A reliable uncertainty is essential for deciding **where** to stop.

Uncertainty-Aware Rollouts

Given: $\mathcal{D}_{\text{env}}, \mathcal{D}_{\text{mod}}, \tilde{p}_{\theta_{1..E}}, \pi, \zeta, \xi, T_{\text{max}}$
 $s_0 \sim \mathcal{U}(\mathcal{D}_{\text{env}})$
for $t = 0, \dots, T_{\text{max}} - 1$ **do**
 $e_t \sim \mathcal{U}(1..E), a_t \sim \pi(\cdot | s_t)$
 $s_{t+1} \sim \tilde{p}_{\theta_{e_t}}(\cdot | s_t, a_t), r_{t+1} = r(s_t, a_t)$
 compute $u_{\text{GJS}}(s_t, a_t)$; **if** $t = 0$: update κ
 if $u_{\text{GJS}}(s_t, a_t) < \kappa$ **then**
 $\mathcal{D}_{\text{mod}} \leftarrow \mathcal{D}_{\text{mod}} \cup \{(s_t, a_t, r_{t+1}, s_{t+1})\}$
 else
 break ▷ stop here, discard
 end if
end for

Algorithm 2: MACURA's rollout

A self-tuning threshold κ

- Each round, the ζ -quantile ($\zeta = 95\%$) of the **first-step** uncertainties = “what counts as certain now”; $\kappa = \frac{\xi}{K} \sum_{k=1}^K \hat{u}_{\text{GJS},k}$.
- $T_{\text{max}} = 10$ and ζ are fixed; only ξ is tuned per task.

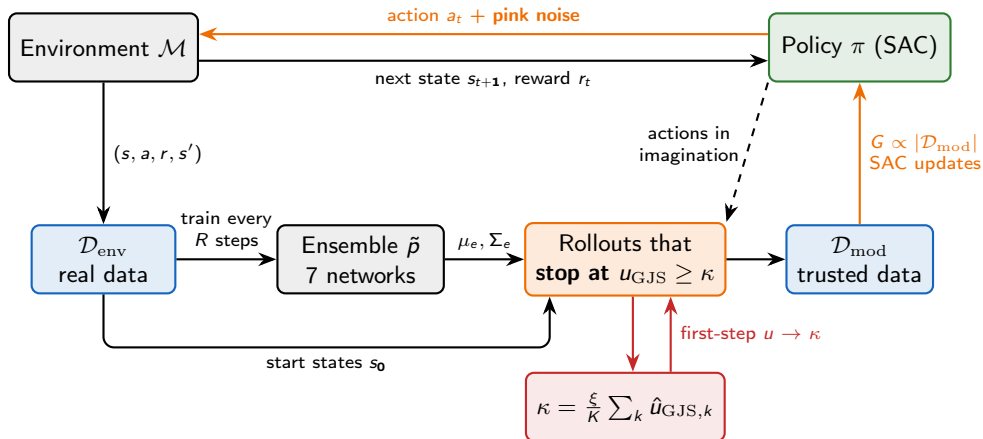
Updates follow the trusted data

$$G = \left\lceil G_{\text{max}} \frac{|\mathcal{D}_{\text{mod}}|}{|\mathcal{D}_{\text{mod}}|_{\text{max}}} \right\rceil$$

Short rollouts (poor model) $\Rightarrow$ few updates; longer ones $\Rightarrow$ more.

Exploration with pink noise (correlated in time) collects informative data and grows $\mathcal{E}$.

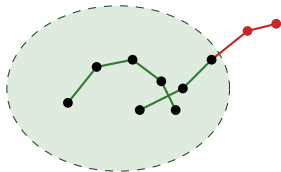
MACURA Architecture



The loop is MBPO's. New in MACURA (**orange/red**): rollouts stop where the model is uncertain, a self-tuning κ , adaptive updates G , pink-noise exploration.

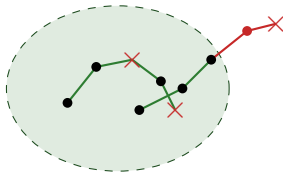
Which Imagined Data Are Kept?

MBPO



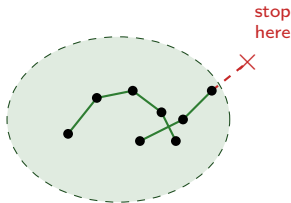
$\mathcal{E}$: model accurate
fixed length

M2AC



$\mathcal{E}$: model accurate
fixed length + masking

MACURA



$\mathcal{E}$: model accurate
stop at the border of $\mathcal{E}$

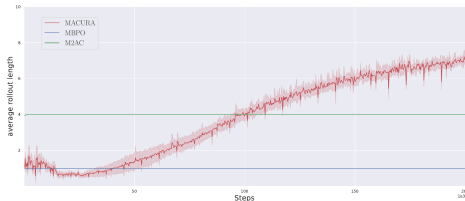
- **MBPO**: same length for every rollout; imagined steps **outside** $\mathcal{E}$ (red) are used.
- **M2AC**: fixed length, then a **fixed share** of the most uncertain steps is removed ($\times$), wherever they are.
- **MACURA**: each rollout **stops at its first uncertain step**: long inside $\mathcal{E}$, nothing outside.

The Four Algorithms Side by Side

	SAC	MBPO	M2AC	MACURA
world model	none	7-network ensemble	same	same
rollout length	–	fixed k , schedule by hand	fixed horizon H	adaptive: until $u_{\text{GJS}} \geq \kappa$ ($\leq T_{\text{max}}$)
imagined data used	–	all of it	a fixed share, least uncertain	only steps inside $\mathcal{E}$
uncertainty	–	none	OvR (one member vs rest)	GJS (all pairs)
stored reward	r	r	$r - \alpha u$	r
SAC updates per real step	fixed	fixed G	fixed G	$G \propto \mathcal{D}_{\text{mod}} $ (adaptive)
exploration (paper)	samples its policy	deterministic	deterministic	pink noise
tuned per task	–	the length schedule	masking, penalty α	only ξ

All three model-based methods share the SAC learner, the ensemble and the 95%/5% data mix: they differ **only in which imagined data reach the learner**.

How Long Are the Imagined Trips in Practice?



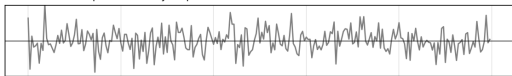
Average rollout length on Walker (paper, Fig. 17).

- **MBPO**: length 1 all training long (its tuned schedule for Walker).
- **M2AC**: a **constant** average, because a fixed share is always kept.
- **MACURA**: **short** while the model is poor (even below 1: it can drop the first step), then growing to about 7 as the model improves.

MACURA's updates follow the same curve: few at first, then more than MBPO's fixed number. The paper checks that **more updates alone** do not explain the gain: MBPO and M2AC with 40 updates per step stay behind.

Noise 1: Exploration Noise

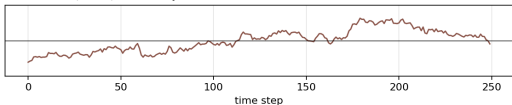
white noise: independent every step



pink noise ($1/f$): correlated in time



random walk (brown): drifts away

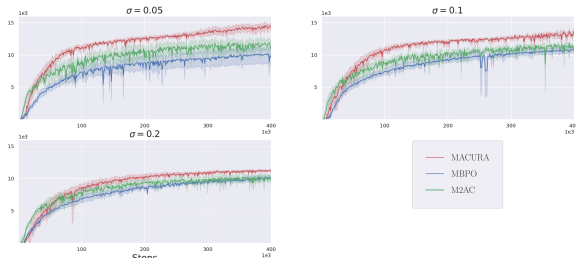


Three kinds of noise added to the actions (same scale).

- **Deterministic**: the policy's mean action, no noise (MBPO and M2AC in the paper).
- **SAC's own**: samples its Gaussian policy, like white noise with a learned width.
- **White noise**: independent every step; the pushes cancel out, so the agent barely leaves the states it already knows.
- **Pink noise ($1/f$)**: **correlated in time**, it pushes in one direction for a while and reaches **new states**, without drifting away like a random walk (Eberhard et al., 2023).

Why it matters for MACURA: new real data at the border of $\mathcal{E}$ make the model accurate there
 $\Rightarrow \mathcal{E}$ grows $\Rightarrow$ longer trusted rollouts.

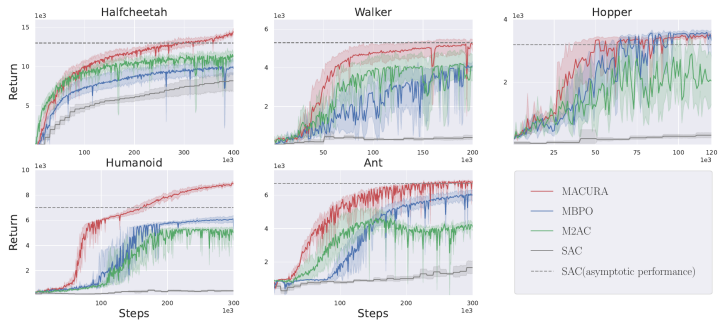
Noise 2: Noise in the Environment



HalfCheetah with hidden action noise $\sigma \in \{0.05, 0.1, 0.2\}$ (paper, Fig. 16).

- **Process noise:** the world itself is random. The paper adds Gaussian noise to every action, and the agent does not see it.
- A model can learn noise only as a **spread**: each member predicts a wide Gaussian (**aleatoric**).
- GJS compares members: if all predict the **same** wide Gaussian, $u_{GJS} \approx 0$ and the step is still trusted. MACURA stops for **disagreement** (epistemic), not for noise.
- MACURA stays ahead of MBPO and M2AC at all three noise levels; the gap shrinks as the noise grows.

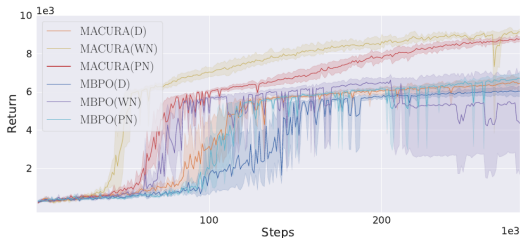
Results on MuJoCo



Return while learning on five MuJoCo tasks; mean of 5 seeds with 95% CI (paper, Fig. 4).

- MACURA learns **faster** and ends **higher** than MBPO and M2AC, most clearly on Humanoid and Ant.
- It often reaches model-free SAC's final return with a fraction of the real data.
- Why: long rollouts where the model is reliable, none where it is not $\Rightarrow$ **more and cleaner** data.

Exploration Changes the Results

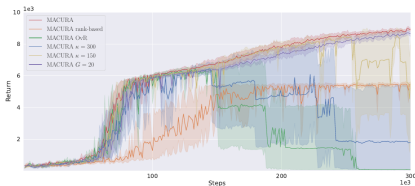


Humanoid: deterministic (D), white noise (WN) and pink noise (PN) exploration (paper, Fig. 5).

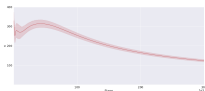
- **Deterministic**: limited but **stable**, for both methods.
- **White noise**: can be very strong for MACURA, but it **destabilises** MBPO.
- **Pink noise**: the best balance of performance and stability.
- MACURA gains the most from better exploration: new data grow $\mathcal{E}$, exactly where it uses the model.
- With deterministic interaction for **all** methods, MACURA is still the strongest on most tasks (paper appendix).

Note: the main comparison gives pink noise only to MACURA; I kept this protocol in my project.

What Matters? Ablations and the Threshold



Humanoid, one building block changed (paper, Fig. 7).



κ during training (paper, Fig. 8).

- **Rank-based** masking (M2AC's rule) instead of a threshold: much worse.
- **OvR** uncertainty instead of GJS: **diverges**, unreliable data get through.
- A **fixed** κ : unstable at some stage of training. The adaptive κ starts high (rough data are enough at first) and then drops (precise data are needed later): a schedule that **emerges by itself**.
- A **fixed** number of updates: a moderate loss.
- ξ works over a **wide range**; too large exploits model errors, too small overfits.

The Paper in Short

Takeaways

- Change the question from *when* to *where* to trust the model.
- Rollouts inside a set $\mathcal{E}$ of accurate states $\Rightarrow$ monotonic improvement.
- GJS: a cheap, closed-form, reliable measure of disagreement (not of noise).
- Rollouts stop at the first uncertain step; κ and the updates G tune themselves.
- One knob, ξ ; better data efficiency than MBPO and M2AC.

Limits

- ξ still differs a lot between tasks (0.3 to 30).
- Very long training destabilises MBPO and MACURA alike.
- The main comparison uses **different exploration** (pink noise vs deterministic).
- Only simulated locomotion tasks.

My question: does MACURA's advantage hold on a **new task I built myself**, with the same compute for every method, and does it really trust its model less **where the model is wrong**?

Part II

My Project: MACURA Races a Drone

A new task, four algorithms, a fair comparison

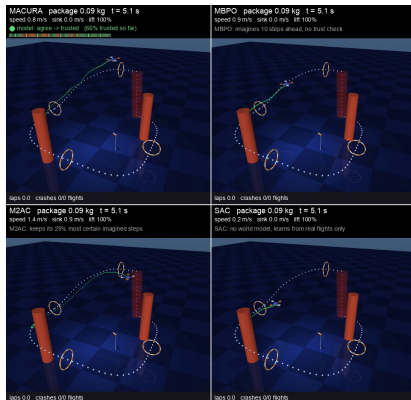
The Project

Goal: I compare the four algorithms of the paper on a task where I **know** where a learned model will be wrong.

- **Why a drone:** real flights are costly and a crash breaks it $\Rightarrow$ data efficiency matters.
- **Same for all:** SAC learner, ensemble, data mix, number of real steps, evaluation scenarios. **Only the rollout rule differs.**
- **MACURA** vs **MBPO** (the method it builds on), **M2AC** (the closest one) and **SAC** (no model).

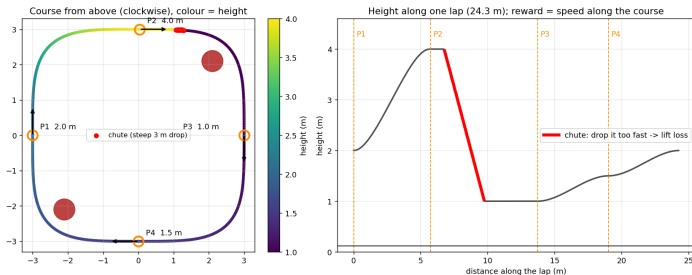
Expectation

- MACURA learns faster with the same compute.
- It trusts its model **less** where the model is wrong.



The four trained drones on the same new scenario.

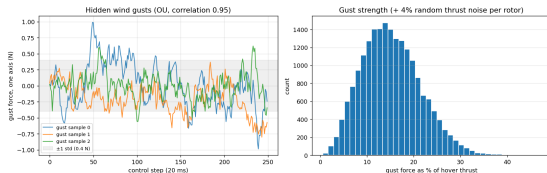
The Task: Race3



The course from above and its height profile. Red: the chute, a steep 3 m drop.

- A quadrotor (MuJoCo) flies laps through **4 gates** (2, 4, 1, 1.5 m high) around **2 pillars**.
- One lap = 24.3 m; a flight lasts **10 s**.
- **4 actions** at 50 Hz (sideways acceleration x , y , climb, turn); **27 observations**.
- **Reward**: speed along the racing line; a crash costs 500 and ends the flight.

The Noises in My Task



Hidden gusts: three samples over 5 s, and their strength.

Environment noise (like the paper's noisy HalfCheetah)

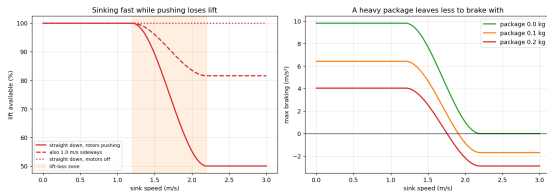
- **Hidden gusts:** random, correlated in time, typically 10–20% of the hover thrust.
- **Rotor noise:** each rotor's thrust varies by 4%.
- **Visible:** a steady wind and a package (0–0.2 kg).

Exploration noise (as in the paper)

- MACURA: **pink noise** on its actions; MBPO and M2AC: none; SAC: samples its policy.

The gusts make every prediction a little uncertain (aleatoric): MACURA should **not** stop for that, only where its members **disagree**.

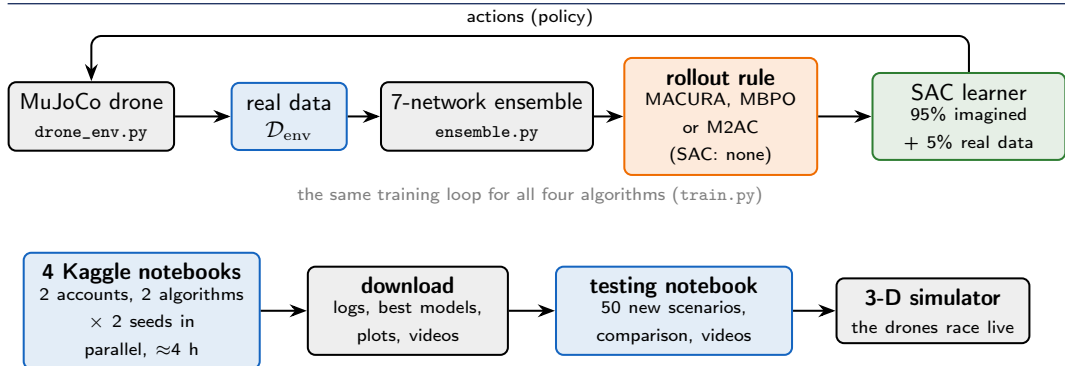
The Chute Trap: Where the Model Should Disagree



Sinking fast while pushing loses lift; a heavy package leaves less to brake with.

- In the chute the drone must drop 3 m almost vertically.
- Sinking faster than **1.2 m/s** while upright and pushing loses up to **50%** of the lift (fully lost at 2.2 m/s) $\Rightarrow$ it cannot brake and crashes.
- This is **rare** in the data and **hard to predict** $\Rightarrow$ the members should **disagree** there (epistemic).
- Exactly where MACURA should **stop imagining**, and where MBPO would keep learning from wrong predictions.

Implementation



- Python, PyTorch, MuJoCo, Stable-Baselines3; every setting in one file (`config.py`).
- The notebooks download the code from GitHub; `kaggle.sh` starts, checks and downloads all four.
- Runs are **deterministic** for a given seed: every result can be reproduced.

A Fair Protocol (Fixed Before the Runs)

setting	value
real steps per run	50,000 (first 5,000 random)
imagined trips	25,000 every 250 steps, ≤ 10 steps
SAC batch	95% imagined + 5% real
MACURA	$\xi = 0.5$, $\zeta = 0.95$, $G_{\max} = 16$
MBPO	length 1 $\rightarrow$ 10, 12 updates/step
M2AC	paper rule, 12 updates/step
seeds	4, 5, 6, 7 (new)
test	50 new scenarios (3000–3049)

- **Equal compute:** MBPO and M2AC get 12 SAC updates per real step; MACURA adapts its own and averaged **10.9**.
- The best checkpoint is chosen on other scenarios (100–119): **nothing is tuned on the test**.
- Scores: IQM over seeds, and **seed-by-seed** comparisons (same scenarios, wind and gusts).
- Exploration as in the paper: MACURA pink noise, MBPO and M2AC none, SAC samples its policy.

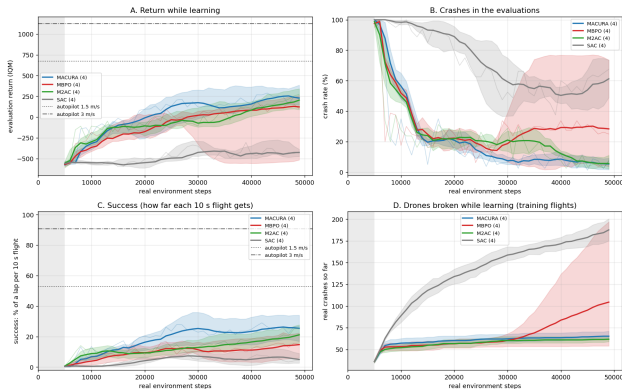
How I Arrived at This Task

task	what happened
cage, delivery	landing tasks: the models were not more uncertain where it mattered, or the drones just hovered
race	laps with a chute: nobody learned to fly in a 10k-step test
race2	+ altitude-hold stabiliser and a racing-line sensor: learnable, but MACURA $\approx$ MBPO
race3	race2 with a deadlier chute (50% lift loss instead of 35%)

- **Rule fixed before the test:** keep race3 only if MACURA beats MBPO while learning in at least 3 of 4 seeds (0–3). It passed: 3 of 4.
- Then the **final study on new seeds** (4–7), which played no part in choosing the task.
- Lesson: a task separates the methods only if the model is uncertain **where it matters**.

Learning Curves

MACURA vs MBPO vs M2AC vs SAC (IQM / mean over seeds, shaded = 95% bootstrap CI; A-C: rolling mean over 7 evaluations, faint = raw; grey = random warm-up; (n) = seeds)



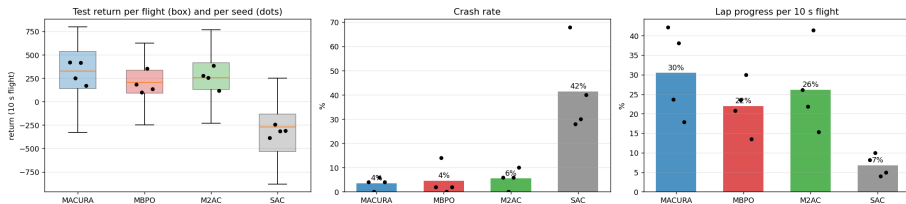
- **A:** MACURA is ahead from about 17k steps and stays ahead.
- **B:** its crash rate falls to about 5% (like M2AC); **MBPO's** rises again after 30k (one seed collapsed).
- **C:** it flies the largest share of a lap.
- **D:** SAC (no model) breaks by far the most drones.

IQM over 4 seeds, 95% CI shaded; dotted lines: a hand-written autopilot.

Test on 50 New Scenarios

	return while learning	drones broken	test return	lap progress	crash rate
MACURA	69	28	333	31%	4%
M2AC	-28	26	267	24%	6%
MBPO	-59	28	158	22%	2%
SAC	-475	154	-312	7%	35%

Test on 50 new scenarios (best checkpoint of every seed)



Each seed's best model on the same 50 new scenarios (dots = seeds). IQM over seeds 4-7.

Seed by Seed

test return	seed 4	seed 5	seed 6	seed 7	MACURA better in
MACURA – MBPO	+238	+151	+62	+40	4 of 4 (mean +123)
MACURA – M2AC	+142	–4	+34	+56	3 of 4 (mean +57)
MACURA – SAC	+807	+492	+731	+483	4 of 4 (mean +628)

All 8 seeds, MACURA vs MBPO

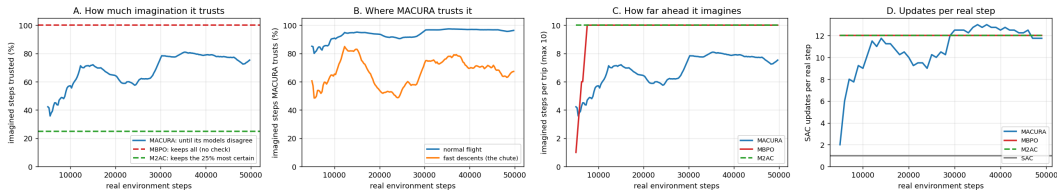
Seeds 0–3 (the race3 test) + 4–7:

- better while learning in **7 of 8** seeds (mean +110; one-sided sign test $p = 0.035$);
- better on the final test flights in **7 of 8** (mean +105; paired t -test $p = 0.035$).

- Same seed = same scenarios, wind and gusts: a **paired** comparison.
- **MBPO seed 5 collapsed** at about 25k steps (return +50 $\rightarrow$ –550, 205 drones broken): it trusts all of its imagination. MACURA had no such collapse.
- M2AC is the strongest baseline: close to MACURA in seed 5.

Does MACURA Trust Its Model Where It Should?

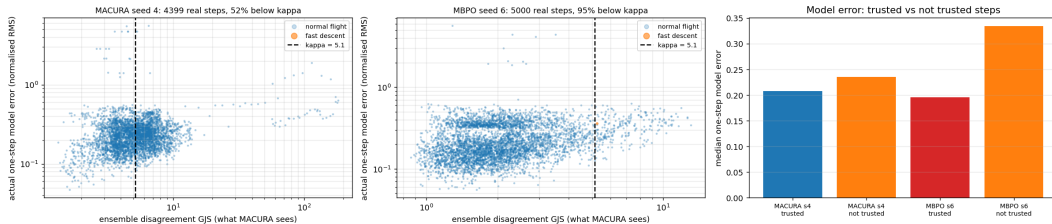
Does MACURA trust its world model 100%? (mean over seeds, rolling mean over 8 model rounds)



- **A:** MACURA trusts about 70% of a full imagination; MBPO 100%, M2AC a fixed 25%.
- **B:** in normal flight it trusts **94%** of imagined steps, in fast descents only **67%**.
- **C:** its trips grow from 4 to about 8 steps as the model improves.
- **D:** its updates follow its trust (Eq. 22).
- 13% of MBPO's imagined data would have failed MACURA's check.

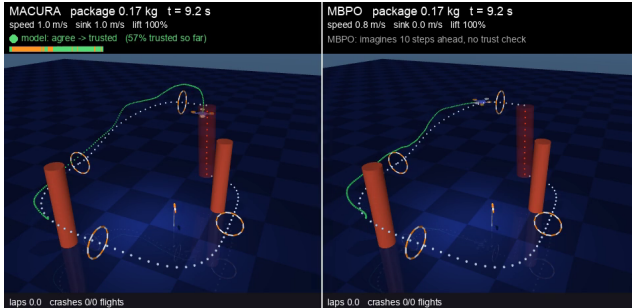
Is the Disagreement Right?

Does the disagreement flag the steps where the world model is actually wrong?



- On real flights I compare each step's disagreement with the model's **real** one-step error (like the paper's Fig. 10).
- Steps that are not trusted have a **higher real error** (MBPO's ensemble: 0.34 vs 0.20; MACURA's: 0.24 vs 0.21): the signal works, but it is **noisy**.
- The trained drones rarely sink faster than 1.2 m/s in the test: MACURA wins by **learning faster and flying further**, not by surviving the chute better.

In Flight



The best MACURA and MBPO models on a new scenario, 9 s into the flight.

- MACURA (left) is already at the chute; MBPO (right) has only reached the top gate.
- MACURA's panel: at every step, would its own model **trust itself** here? (green: members agree; orange: stop imagining).
- Best seeds on the test: MACURA 422, M2AC 383, MBPO 355.
- Videos of every model and a live 3-D simulator are in the repository.

The Experiments I Ran

experiment	task	algorithms	seeds $\times$ steps	what I learned
task screening	cage, delivery, race	mostly MACURA, MBPO	short pilots	the task decides: these could not separate the methods
race2 pilot	race2	all four	seed 0 $\times$ 50k	MACURA best (test 284 vs MBPO 111), but one seed
quick check	race2, race3	MACURA, MBPO	seed 0 $\times$ 10k	pipeline works; runs are exactly reproducible
race3 test	race3	MACURA, MBPO, SAC	seeds 0–3 $\times$ 50k	decision rule passed: 3 of 4 seeds
final study	race3	all four	seeds 4–7 $\times$ 50k	the results in this talk
new-scenario test	race3	16 trained models	50 scenarios each	test tables, trust, videos

- **Compute:** 4 Kaggle GPU notebooks on 2 accounts at once, about 4 h each; the test: 11 min on my laptop.
- **Equal updates** fixed after the race2 pilot: 12 per step for MBPO, MACURA's average there.

Limitations (1/2): Training Budget and Statistics

Few training steps

- Only **50,000 real steps** per run (5,000 of them random); the paper uses 125k–400k.
- The curves are **still rising** at the end: I compare **early learning**, not final performance.
- The task is **not solved**: 31% of a lap per 10 s, against 54% for a simple autopilot at 1.5 m/s.
- Kaggle limits (12 h per session, about 30 GPU h per week and account) did not allow longer runs.

Few seeds

- 4 seeds per algorithm in the final study (8 for MACURA vs MBPO).
- 7 of 8 is **consistent**, but only at the edge of statistical significance ($p \approx 0.035$).
- The confidence intervals are wide; M2AC and MACURA overlap.
- **No hyperparameter search**: $\xi = 0.5$, MBPO's length 1 $\rightarrow$ 10 and M2AC's penalty were set once, not tuned per method.

Limitations (2/2): Fairness and the Task

Fairness

- **Exploration differs:** only MACURA uses pink noise (the paper's protocol). Part of its gain may come from exploration; the paper shows pink noise helps MBPO too.
- MBPO and M2AC use the paper's rules as they are; the paper tunes MBPO's length schedule per task, I did not.

The task

- **One task**, designed and changed by me five times: a risk of picking a task that suits MACURA (mitigated by rules fixed in advance and new seeds).
- The trained drones **rarely reach the chute trap**, so the “where” mechanism was tested less than planned.
- The disagreement predicts the real model error only **weakly** here.
- **Simulation only:** nothing was flown on a real drone.

Lessons and Next Steps

Lessons

- The task decides everything: MACURA only helps if the model is uncertain **where it matters**.
- Fix the decision rules **before** the runs, and confirm on **new seeds**.
- Give every method the **same compute** (updates per step), or the comparison says nothing.
- Separate the two noises: exploration noise changes the data, environment noise the model.

Next steps

- Train **longer** (100k–200k steps), so the drones fly the chute fast and the trap matters.
- **More seeds** (10 or more) for clear statistics.
- The **same exploration** for all methods (already possible in the code).
- Tune ξ , and compare with the paper's settings.
- Test the best policies on a **real drone**.

Conclusion

- **The paper:** MACURA trusts its learned model only **where it trusts itself**. Rollouts stop at the first uncertain step ($GJS \geq \kappa$), κ and the updates tune themselves, and one knob (ξ) is left. GJS reacts to disagreement, not to noise.
- **My project:** on a racing drone with wind, gusts, a package and a chute, with the same compute for all:
 - MACURA had the **best test return** (333 vs 267 M2AC, 158 MBPO, -312 SAC);
 - it beat MBPO in **7 of 8 seeds** and M2AC in 3 of 4, with no collapse;
 - it trusted its model **less in fast descents** (67%) than in normal flight (94%).
- With only 50k steps and few seeds this is **early-learning evidence**: consistent, but not yet proof.
- Model-based RL is far more data-efficient than model-free SAC here, and MACURA was the strongest of the three model-based methods on this task.

Thank you for your attention!

Code, results and videos: github.com/silvergjecha22/macura-drone